

NovaSlim: Practical Folding Proofs with Slim On-Chain Proofs and Post-Quantum Commitment Modularity

Pawel Jakubas

Cardano Foundation, Switzerland

Abstract. We present NovaSlim, a Nova-family IVC system that produces **sub-kilobyte transparent proofs** (the slim proof is $\sim 0.4\text{--}1.5$ KiB, depending on circuit size; the total on-chain payload including the NIFS bundle is $\sim 0.8\text{--}2.5$ KiB for Pedersen and Hash, and $\sim 10\text{--}12$ KiB for SIS $m = 128$) with no trusted setup on a single standard curve. NovaSlim makes three contributions: (1) *slim proofs*, which strip polynomial commitment openings from the on-chain payload while preserving a formal knowledge-soundness guarantee; (2) *commitment-scheme modularity*, a generic trait with three instantiations — Pedersen (classical), SIS/Ajtai (lattice-based, up to $14\times$ faster folding on BLS12-381), and Hash (on-the-fly Blake2b, zero param storage) — two of which are post-quantum; and (3) the empirical demonstration that these techniques combine into a practical system. NovaSlim produces slim proofs of $\sim 0.4\text{--}2.5$ KiB that verify in $0.1\text{--}1.0$ ms — all without a curve cycle or trusted setup. The system is implemented in Rust with support for six elliptic curves (BLS12-381, BN254, Grumpkin, Bandersnatch, Pallas, Vesta) and three commitment schemes (Pedersen, SIS, Hash).

Keywords: folding proofs · IVC · succinct arguments · post-quantum cryptography · commitment schemes · blockchain verification

1 Introduction

1.1 The problem

Zero-Knowledge Succinct Non-interactive ARguments of Knowledge (zkSNARKs) such as Groth16 [Gro16] have become the standard tool for on-chain privacy and scalability. However, they suffer from three well-known limitations:

1. **Trusted setup.** Groth16 requires a per-circuit trusted setup ceremony. For circuits with millions of constraints this ceremony can take minutes or hours and produces proving keys of comparable size.
2. **Memory scaling.** The prover’s memory consumption is $O(C)$ where C is the total constraint count. Circuits exceeding ~ 4 M constraints can exhaust commodity RAM even with sparse-matrix optimizations.
3. **Proof size vs. verifier cost trade-offs.** Groth16 proofs are tiny (192 bytes) but require a pairing check. STARKs avoid pairings but have larger proofs. Recursive SNARKs combine both but embed an expensive verifier circuit at every step.

Nova [KST22] addresses (2) and partially addresses (1) by splitting a computation into N identical *step circuits* and *folding* them into a single accumulator instance using a

Relaxed-R1CS scheme. Folding is transparent: the prover only performs two $O(|\mathbb{F}|)$ -sized multi-scalar multiplications per step, and the accumulator is constant-size regardless of N . The catch is that the folded instance must still be *compressed* into a final SNARK; the original Nova paper uses Spartan [Set20] for this, which is also transparent.

However, Nova assumes a *cycle of elliptic curves* (e.g. BN254/Grumpkin). The primary curve hosts the R1CS constraints; the secondary curve hosts the Pedersen commitments that are verified *in-circuit* at each fold. This makes Nova incompatible with blockchains that have standardized on a *single* pairing-friendly curve — for example Cardano (BLS12-381) and, to a lesser extent, Ethereum (BN254), where on-chain precompiles exist only for the native curve.

1.2 Our contributions

NovaSlim combines existing techniques from Nova, CycleFold, and Spartan with three novel contributions that together make the system practical for single-curve, size-constrained blockchains:

1. Slim proofs (§5). We prove that HashPC opening proofs (truth tables) can be stripped from the on-chain payload without compromising knowledge soundness. The sumcheck protocol already binds the prover to a valid witness; the openings are only needed for an off-chain audit trail (a separate record that allows an auditor to verify the full proof independently). This reduces the on-chain proof from ~ 240 KiB to $\sim 1\text{--}1.5$ KiB for medium-scale circuits (e.g., Ed25519, 7,724 constraints) and down to ~ 0.4 KiB for tiny circuits (e.g., VRF, 9 constraints) — a $160\text{--}600\times$ reduction — while preserving a formal security guarantee (Theorem 1). To our knowledge, this is the first demonstration that polynomial commitment openings can be dropped from a folding-scheme proof while retaining a rigorous knowledge-soundness argument.

2. Commitment-scheme modularity (§2 and §6). We abstract the folding commitment behind a generic `CommitmentScheme` trait with three instantiations: *Pedersen* (deterministic elliptic-curve MSM, classically secure), *SIS/Ajtai* (lattice-based matrix-vector multiplication, $\sim 8\times$ faster folding), and *Hash* (on-the-fly Blake2b coefficient derivation, zero param storage). Two of the three paths (SIS and Hash) are post-quantum. The security proof (Theorem 1) is generic over the commitment scheme: only the binding property is used, so the same slim proof format and verifier work with all three instantiations. This is, to our knowledge, the first Nova-family implementation where the commitment scheme is a runtime-selectable parameter rather than a protocol-level choice.

3. The combination and its empirical evaluation (§6). We assemble off-circuit folding (as in CycleFold [KS23]), sumcheck+HashPC compression (building on Spartan [Set20]), and the slim-proof optimization into a single pipeline, and evaluate it end-to-end on a real circuit (Ed25519 verify, 7,724 constraints, 255 steps). The combination is not individually novel — each technique exists in prior work — but the resulting system achieves a property that no prior Nova-family system has demonstrated: **sub-kilobyte transparent proofs with no trusted setup and two independent post-quantum commitment paths, all on a single standard curve.**

Design choices. Off-circuit folding (§3) sacrifices the IVC “proof-carrying” property at intermediate steps: each fold is not proved until the final compression. This is acceptable for our target use case — batch proving a long sequential computation where the prover is trusted until the final proof. The benefit is that no curve cycle is needed: the system works on any curve with a hard DLOG problem and efficient MSM (BLS12-381, BN254, Pallas, Vesta). The sumcheck compression (§4) replaces Spartan’s IPA or KZG commitments with HashPC, which is transparent and avoids the need for a curve cycle or trusted setup. This is simpler than Spartan but not more efficient for large circuits; our target is $k \leq 13$ (up to $\sim 8K$ constraints).

Paper roadmap. §2 reviews the cryptographic building blocks (relaxed R1CS, sumcheck, HashPC). §3 describes off-circuit folding on a single curve. §4 explains sumcheck compression. §5 proves the slim-proof security guarantee (Theorem 1). §6 evaluates the implementation. §7 concludes.

1.3 Related work

Nova and successors. The original Nova [KST22] uses a curve cycle and Spartan compression. SuperNova [KS22] generalizes to non-uniform steps. HyperNova [KS24] supports customizable constraint systems. CycleFold [KS23] reduces the cycle requirement but still needs a secondary curve. All of these assume either a full cycle or CycleFold’s relaxed requirement. Our work shows that off-circuit folding on a single standard curve is a viable alternative when recursive proof-carrying is not required.

Halo and recursive proof composition. Halo [BGH19] was the first practical realization of recursive proof composition without a trusted setup, using discrete-log cycles of ordinary (non-pairing-friendly) curves and an amortized inner-product argument. Halo2, the successor framework implemented by the Zcash team, replaces the inner-product argument with a PLONKish arithmetization and polynomial commitments via IPA. Both Halo and Halo2 require a curve cycle. NovaSlim is complementary: it does not achieve recursive proof composition (folding is off-circuit), but it produces constant-size proofs without a curve cycle and without a trusted setup.

Sonobe and other folding libraries. Sonobe [0xP24] is a modular Rust library implementing Nova, HyperNova, ProtoGalaxy, and CycleFold with multiple deciders (Spartan, Groth16, KZG) and frontends (arkworks, Circom, Noir). Its focus is EVM on-chain verification: the recommended path uses a curve cycle (e.g. BN254/Grumpkin) and a Groth16 decider for Ethereum smart contracts. This makes Sonobe the right choice for EVM rollups and account-based chains that support smart contracts and can absorb a 192-byte Groth16 proof. NovaSlim is complementary: it targets UTXO ledgers with hard transaction-size limits (e.g. Cardano’s 16 KiB) where Groth16 is unavailable and even Spartan proofs are too large. By folding off-circuit and stripping HashPC openings, NovaSlim achieves $\sim 0.4\text{--}1.5$ KiB transparent proofs (depending on circuit size) with no trusted setup and sub-millisecond verification on a single standard curve.

Axiom and production zkVMs. Axiom is a production system that uses Halo2-based circuits to verify historical Ethereum data on-chain. Their OpenVM framework (successor to the ZK coprocessor) is a modular RISC-V zkVM that generates SNARK proofs for arbitrary Rust programs, with a proving API that achieves real-time Ethereum block proving on GPU clusters. Axiom’s approach is SNARK-based (PLONKish + FRI), not folding-based, and targets Ethereum’s account model with its EVM precompiles. NovaSlim is orthogonal: we target UTXO blockchains with hard transaction-size limits, use folding rather than monolithic SNARKs, and produce proofs that verify in sub-millisecond without precompilation.

Spartan and sumcheck-based SNARKs. Spartan [Set20] uses the sumcheck protocol over multilinear polynomials with IPA commitments. Our compression also uses sumcheck, but over the Boolean hypercube directly on the relaxed R1CS products, paired with HashPC rather than IPA or KZG. This is simpler and avoids the need for polynomial-evaluation arguments on a curve cycle.

Lattice folding. LatticeFold [BC24] is the first lattice-based folding scheme, replacing the Pedersen commitment in Nova with an Ajtai commitment over Module-SIS. The key challenge is ensuring that extracted witnesses remain low-norm after many rounds of folding; LatticeFold addresses this with a sumcheck-based range proof using bit decomposition. LatticeFold+ [BC25] improves on this with a purely algebraic range proof that avoids bit decomposition entirely, achieving 5–10 $\times$ faster folding. Both schemes operate over small (64-bit) fields and achieve post-quantum security under Module-SIS. NovaSlim’s SIS

instantiation is simpler: we use a direct Ajtai commitment without the norm-preservation machinery, because our off-circuit folding does not need to prove the fold itself. The trade-off is that our SIS instantiation does not provide the same formal security guarantees as LatticeFold for recursive IVC; it is designed for batch-proving scenarios where the prover is trusted until the final proof. The Hash instantiation (Blake2b-based, zero param storage) has no analogue in LatticeFold.

Comparison table. Table 1 summarizes the key differences between NovaSlim and related systems.

Table 1: Comparison of NovaSlim with related systems.

System	Curve cycle	Proof size	Setup	Folding	PQ path	On-chain verify
Nova [KST22]	Required	~10 KiB	None	Recursive	No	Spartan
CycleFold [KS23]	Relaxed	~10 KiB	None	Recursive	No	Spartan
Halo2	Required	~100 KiB	None	Recursive	No	IPA verify
Spartan [Set20]	Not needed	~100 KiB	None	N/A (SNARK)	No	Sumcheck
LatticeFold+ [BC25]	Not needed	~10 KiB	None	Recursive	Yes (SIS)	N/A
NovaSlim	Not needed	~0.4–1.5 KiB	None	Off-circuit	Yes	~0.2 ms

NovaSlim is the only system that combines: (1) no curve cycle, (2) sub-kilobyte proofs, (3) no trusted setup, (4) post-quantum commitment options, and (5) sub-millisecond on-chain verification. The trade-off is that NovaSlim’s off-circuit folding provides a weaker security guarantee than recursive IVC: the prover is trusted during folding, and soundness relies on the final compressed proof (Definition 2).

1.4 Data flow

The NovaSlim pipeline has four stages, with clear separation between prover (off-chain) and verifier (on-chain or audit). Figure 1 shows the data flow and the sizes of artifacts at each stage.

What travels on-chain: only the NIFS bundle (~2 KiB for Pedersen, ~10 KiB for SIS $m = 128$) and the slim proof (~0.4–2.5 KiB, depending on $k = \log_2 n_{\text{constraints}}$). The full sumcheck proof with HashPC openings stays off-chain as an audit trail. The slim verifier needs no proving key, no verifying key, no trusted-setup artifacts, and no circuit file — only the NIFS bundle (which already contains constraint dimensions) and the slim proof.

2 Preliminaries

2.1 Notation

We use λ for the security parameter. $\mathbb{F}$ denotes a finite field (e.g. the scalar field of BLS12-381, BN254, Pallas, Vesta, or Bandersnatch in our implementations). $\mathbb{G}$ denotes a group (e.g. G1 of the pairing-friendly curves or the prime-order group of the pasta / Bandersnatch curves). The symbol $\circ$ denotes the Hadamard (element-wise) product.

2.2 Relaxed R1CS

A relaxed R1CS instance is a tuple $U = (x, u, \bar{W}, \bar{E})$ where x is the public input, $u \in \mathbb{F}$ is the slack scalar (a scaling factor that absorbs the folding error), $\bar{W}$ is a commitment to the witness W , and $\bar{E}$ is a commitment to the error vector E . The instance is satisfied by a witness (W, E) if:

$$(AZ) \circ (BZ) = u \cdot (CZ) + E$$

where $Z = (1, W, x, u)$ and A, B, C are the R1CS constraint matrices.

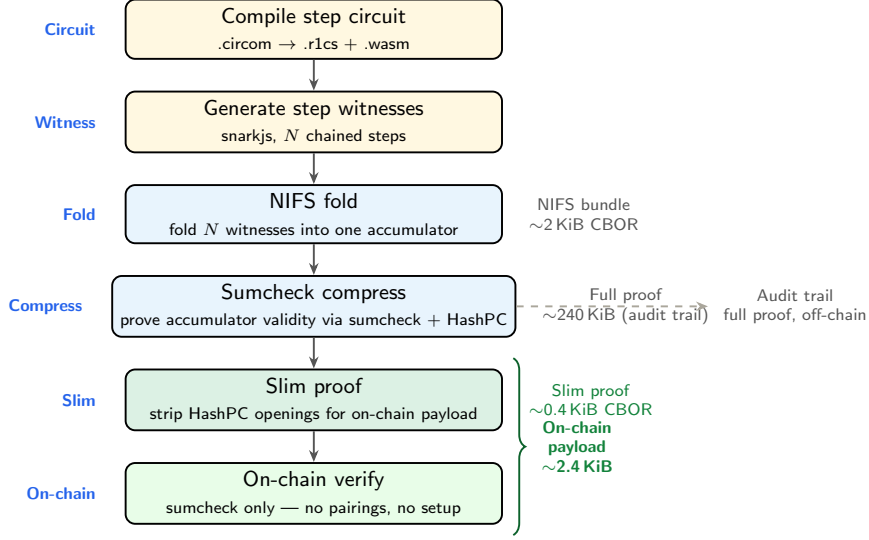


Figure 1: The NovaSlim pipeline. The prover folds N step witnesses into a single accumulator, compresses it via sumcheck + HashPC, then strips the opening proofs for the slim on-chain payload (~ 0.4 – 2.5 KiB total for Pedersen, ~ 10 – 12 KiB for SIS $m = 128$). The full proof (~ 240 KiB) is retained off-chain as an audit trail.

2.3 The sumcheck protocol

The sumcheck protocol [LFKN90] is an interactive proof that a sum over the Boolean hypercube equals a claimed value. For a k -variate polynomial g , the prover convinces the verifier that $\sum_{x \in \{0,1\}^k} g(x) = H$. The protocol has k rounds; in round i , the prover sends a univariate polynomial $h_i(X_i)$ and the verifier replies with a random challenge r_i . The protocol is made non-interactive via the Fiat-Shamir transform.

For our use case, the polynomial is the per-constraint product:

$$P(j) = (AZ)_j \cdot (BZ)_j - u \cdot (CZ)_j - E[j]$$

and the sumcheck proves $\sum_{j \in \{0,1\}^k} P(j) = 0$.

2.4 HashPC: hash-based polynomial commitment

HashPC is a polynomial commitment scheme combining two binding commitments:

1. **Truth-table hash:** $H(tt)$ where tt is the vector of MLE evaluations at all Boolean hypercube points, hashed with a collision-resistant function (we use BLAKE2b-512).
2. **Pedersen commitment:** $\text{Com}(v) = \sum_i v_i G_i$ where G_i are deterministic bases derived from a public seed.

The opening proof is the truth table itself; verification recomputes the hash and checks consistency. This is not succinct in the traditional sense (the opening is $O(2^k)$ field elements), but for the sizes we care about ($k \leq 13$, i.e. up to $\sim 8K$ constraints) it is concretely efficient and transparent.

3 Off-circuit folding on a single curve

3.1 Why Nova needs a curve cycle

In the original Nova IVC scheme, each step circuit is *augmented* with a verifier circuit that checks the previous step’s folding. This verifier circuit must evaluate Pedersen commitments, which are group operations on G_1 . But the circuit itself is expressed over the scalar field $\mathbb{F}_r$ of the primary curve. To make group operations native in $\mathbb{F}_r$, the secondary curve’s base field must equal the primary curve’s scalar field — hence the cycle.

3.2 Off-circuit folding removes the cycle requirement

We take a different approach: folding happens entirely *off-circuit*, as a pre-processing step. The prover maintains the running accumulator (U_{acc}, W_{acc}) in memory and folds each new step instance into it using the NIFS protocol. Only the *final* folded instance is ever proved.

Trade-off: We lose the IVC property that each step produces a standalone proof of correct folding. Intermediate steps have no proof; if the prover is dishonest during folding, the error is only caught at the final compression. For our use case — batch proving a long sequential computation where the prover is trusted until the final proof — this is acceptable.

Benefit: We can use any elliptic curve with a hard DLOG problem. The only requirements are (1) a scalar field $\mathbb{F}_r$ suitable for R1CS, and (2) a G_1 group for Pedersen commitments with efficient MSM. The implementation is generic over the curve: concrete instantiations are provided for BLS12-381 (e.g. Cardano), BN254 / Grumpkin (e.g. Ethereum rollups), Pallas / Vesta (e.g. Zcash Orchard), and Bandersnatch (a fast prime-order curve over the BLS12-381 scalar field).

3.3 The NIFS fold

Our fold is identical to Nova’s NIFS [KST22] (Section 4):

Inputs: two relaxed instances $U_1 = (x_1, u_1, \bar{W}_1, \bar{E}_1)$ and $U_2 = (x_2, u_2, \bar{W}_2, \bar{E}_2)$ with witnesses (W_1, E_1) and (W_2, E_2) .

Fiat-Shamir challenge: $r = H(\text{FOLDPREFIX} \parallel acc \parallel U_1 \parallel U_2)$ where acc is the running transcript hash.

Folded witness:

$$\begin{aligned} W &= W_1 + r \cdot W_2 \\ E &= E_1 + r \cdot E_2 + r \cdot T \end{aligned}$$

where T is the cross-term vector:

$$T = (AZ_1) \circ (BZ_2) + (AZ_2) \circ (BZ_1) - u_1(CZ_2) - u_2(CZ_1)$$

Folded instance:

$$\begin{aligned} x &= x_1 + r \cdot x_2 \\ u &= u_1 + r \cdot u_2 \\ \bar{W} &= \bar{W}_1 + r \cdot \bar{W}_2 \\ \bar{E} &= \bar{E}_1 + r \cdot \bar{E}_2 + r \cdot \text{Com}(T) \end{aligned}$$

The fold is transparent (no trusted setup engaged), linear-time, and produces a relaxed instance that is satisfiable exactly when both inputs are satisfiable.

4 Transparent compression: sumcheck + HashPC

4.1 Why not Spartan?

Spartan [Set20] is an excellent compression SNARK, but it requires polynomial commitment evaluation arguments (IPA or KZG) that are expensive to implement on a single curve without a cycle. IPA requires curve-cycle-friendly inner-product arguments; KZG requires a trusted setup. Both add significant complexity for our target use case.

4.2 Sumcheck over relaxed R1CS

We express the relaxed equation as a sum over the Boolean hypercube. Let $n = 2^k$ be the number of constraints (padded to a power of two). Define the polynomial:

$$P(j) = (AZ)_j \cdot (BZ)_j - u \cdot (CZ)_j - E[j]$$

The sumcheck protocol proves that $\sum_{j \in \{0,1\}^k} P(j) = 0$.

Prover (Algorithm 1). For each round $i = 1 \dots k$:

1. Compute the univariate polynomial $h_i(X) = \sum_{j \in \{0,1\}^{k-i}} P(r_1, \dots, r_{i-1}, X, j)$.
2. Send h_i to the verifier.
3. Receive challenge r_i from the verifier (Fiat-Shamir).

The degree of h_i is at most 1 (the MLE of P is multilinear). In practice we represent h_i by its evaluations at $X = 0$ and $X = 1$.

Verifier (Algorithm 2). For each round i :

1. Check that $h_i(0) + h_i(1)$ equals the previous claim.
2. Compute $r_i = H(\text{transcript} \parallel h_i)$.
3. Update the claim to $h_i(r_i)$.

After k rounds, the verifier holds the final claim $c = P(r_1, \dots, r_k)$. The prover then provides the claimed evaluation $prod_r = P(r)$, and the verifier checks:

$$prod_r = c$$

By the Schwartz-Zippel lemma, if this holds for random r , then P is identically zero with overwhelming probability, implying the relaxed R1CS holds for all constraints.

4.3 HashPC commitments

The prover must convince the verifier that the claimed evaluations az_r, bz_r, cz_r, e_r are consistent with the committed witness W and error E in the folded instance. HashPC provides this binding:

Com(v):

1. Compute the truth table $tt = \text{MLE}(v)$ evaluated at all Boolean hypercube points.
2. Compute $hash = \text{BLAKE2b-512}(tt)$.
3. Compute $ped = \text{Pedersen}(v) = \sum_i v_i G_i$ where $G_i = H(seed \parallel domain \parallel i) \cdot G$.
4. Output $(hash, ped)$.

Open(v): Output the truth table tt .

Verify($hash, ped, tt, r, claimed_eval$):

1. Check $BLAKE2b-512(tt) = hash$.
2. Check $MLE(tt)(r) = claimed_eval$.
3. (Optional) Check $Pedersen(tt[:len(v)]) = ped$.

The Pedersen check is done natively off-circuit; for on-chain verification (our slim proofs), we skip it because the sumcheck protocol already binds the prover to a valid witness.

5 Slim proofs: constant-size on-chain verification

5.1 The problem: opening proofs are large

For a step circuit with $n_{wires} = 7,658$ and $n_{constraints} = 7,724$, the truth tables for W and E have 7,658 and 8,192 entries respectively. Serialized as JSON decimal field elements, the full sumcheck proof is ~ 473 KiB. This, for example, exceeds the 16 KiB transaction size limit of Cardano by $\sim 30\times$.

5.2 The slim-proof optimization

Our key observation is that the HashPC opening proofs (the truth tables) are not needed for *soundness* of the on-chain verification. They are only needed for a separate *audit trail* that convinces an off-chain party that the committed values actually correspond to the claimed evaluations.

Slim proof structure:

- Sumcheck round polynomials (k field elements, where $k = \log_2 n_{constraints}$)
- Fiat-Shamir transcript (the challenge sequence $r_1, \dots, r_k$)
- Claimed product evaluation $prod_r = P(r)$
- Commitment hashes ($hash_W, hash_E$) — 64 bytes each

Total on-chain size: ~ 0.4 – 1.5 KiB depending on $k = \log_2 n_{constraints}$ (with binary serialization). For the Ed25519 circuit ($k = 13$) the slim proof is ~ 1.5 KiB; for the VRF circuit ($k = 4$) it is ~ 0.4 KiB.

5.3 Security argument

We formalize the batch-proving model, define what “soundness” means in this context, and then prove that NovaSlim achieves it.

5.3.1 The batch-proving model

Definition 1 (Batch-proving model). The batch-proving model is a two-phase protocol between a *prover* and a *verifier*:

Phase 1 — Folding (prover only). The prover receives N step instances $(x_1, \dots, x_N)$ with corresponding step witnesses $(W_1, \dots, W_N)$. Using the NIFS protocol, the prover folds these into a single relaxed R1CS accumulator $U_{acc} = (x_{acc}, u_{acc}, \bar{W}_{acc}, \bar{E}_{acc})$. No verification occurs during this phase.

Phase 2 — Compression and verification. The prover compresses U_{acc} via sumcheck + HashPC into a slim proof π . The verifier checks $\text{VerifySlim}(B, \pi) = 1$.

Adversary model. The adversary controls:

- The step witnesses $(W_1, \dots, W_N)$ (can be arbitrary field elements).
- The folding process (can apply NIFS incorrectly or dishonestly).
- The sumcheck messages $h_1, \dots, h_k$ (can attempt to cheat the verifier).

The verifier has access to:

- The NIFS bundle $B = (U_{acc}, \text{Com}_W, \text{Com}_E, \text{transcript_hash})$.
- The slim proof $\pi = (h_1, \dots, h_k, r_1, \dots, r_k, az_r, bz_r, cz_r, e_r)$.
- The step circuit description (to derive constraint dimensions).

Trust assumption. The prover is *untrusted during Phase 1*: it may produce an invalid accumulator. The security guarantee is that the final compressed proof (Phase 2) catches any cheating: either the extractor recovers a valid witness, or extraction fails. This is appropriate for batch proving where the prover is the data owner and the verifier only needs assurance that a valid witness *exists*.

5.3.2 The batch-proving relation

We define the two relations that the slim proof must connect.

Let R_RELAXED denote the relaxed R1CS relation:

R_RELAXED : Given public parameters (A, B, C, n, u, x) , find $(W, E) \in \mathbb{F}^{n_{wires}} \times \mathbb{F}^n$ such that $(A \cdot Z) \circ (B \cdot Z) = u \cdot (C \cdot Z) + E$ where $Z = (1, W, x, u)$ and n is padded to 2^k .

Let H_{cr} denote a collision-resistant hash function (BLAKE2b-512), and let H_{ro} denote a random oracle modelling the Fiat-Shamir transcript hash.

The relation witnessed by a slim proof. A slim proof $\pi = (h_1, \dots, h_k, r_1, \dots, r_k, \text{prod}_r)$ together with a NIFS bundle $B = (U_{acc}, \text{Com}_W, \text{Com}_E, \text{transcript_hash})$ witnesses the relation:

R_SLIM : The verifier $\text{VerifySlim}(B, \pi) = 1$, meaning:

- (SC) The sumcheck protocol accepts: $h_i(0) + h_i(1) = h_{i-1}(r_{i-1})$ for $i = 1 \dots k$, and $\text{prod}_r = h_k(r_k)$.
- (CB) The commitment hashes in B match the Fiat-Shamir transcript: $\text{transcript_hash} = H_{cr}(h_1 \parallel \dots \parallel h_k \parallel r_1 \parallel \dots \parallel r_k \parallel \text{prod}_r \parallel \text{Com}_W \parallel \text{Com}_E)$.

Note that R_SLIM is a *syntactic* relation (only verifier checks — no witness is needed, only the proof structure is verified), while R_RELAXED is a *semantic* relation (requires a witness satisfying the algebraic constraint). The theorem shows that R_SLIM implies R_RELAXED .

Definition 2 (Batch-proving soundness). A batch-proving scheme is *sound* if for any PPT adversary A that produces a verifying slim proof (B, π) with probability $\varepsilon > \text{negl}(\lambda)$, there exists a PPT extractor E that outputs a witness (W, E) satisfying R_RELAXED with probability $\varepsilon - \text{negl}(\lambda)$.

This definition captures the core guarantee: a valid slim proof implies the *existence* of a valid witness. The extractor E may be inefficient (e.g., require $O(2^k)$ rewinding queries), but the *soundness property* itself is computational — no PPT adversary can produce a valid proof without “knowing” a valid witness in the sense that the extractor can recover it.

5.3.3 Proof of Theorem 1

Assumptions:

- **(A1) Sumcheck knowledge-soundness.** The sumcheck protocol over the Boolean hypercube is a proof of knowledge: there exists a PPT extractor Σ that, given oracle access to a prover P^* that produces a valid sumcheck transcript, outputs a function $f : \{0, 1\}^k \rightarrow \mathbb{F}$ such that f is the multilinear extension of some $v \in \mathbb{F}^{2^k}$ with $\sum_{x \in \{0, 1\}^k} f(x) = \text{claimed_sum}$, and f agrees with the prover's claimed final evaluation at the random point r [LFKN90].
- **(A2) Fiat-Shamir soundness.** The Fiat-Shamir transform preserves the knowledge-soundness of the sumcheck protocol in the random oracle model [BSBHR18]. Concretely, if the random oracle queries are consistent with a valid sumcheck transcript, the extractor Σ still applies.
- **(A3) Transcript binding.** H_{cr} is collision-resistant: finding $m \neq m'$ with $H_{cr}(m) = H_{cr}(m')$ is computationally infeasible.

Note on commitment binding. The commitment scheme Com is computationally binding (under DLOG for Pedersen, SIS for SIS, and Blake2b collision resistance for Hash). This property is essential for the *full* proof (audit trail), which checks that the opening truth tables correspond to the committed witness. The slim-proof soundness theorem below does *not* require commitment binding: the sumcheck extractor alone guarantees that a valid witness exists (Definition 2).

Theorem 1 (NovaSlim achieves batch-proving soundness). *Assume (A1)–(A3). Then NovaSlim's slim-proof verification satisfies Definition 2: there exists a PPT extractor E that, given oracle access to a prover P^* that produces a verifying slim proof (B, π) with probability $\varepsilon > \text{negl}(\lambda)$, outputs a witness (W, E) satisfying R_RELAXED with probability $\varepsilon - \text{negl}(\lambda)$.*

Proof. We proceed by a sequence of games.

Game 0. The real game. Prover P^* outputs (B, π) and the verifier checks $\text{VerifySlim}(B, \pi) = 1$. Let $\varepsilon_0 = \varepsilon$ be the success probability.

Game 1. Replace the random oracle H_{ro} used in the Fiat-Shamir transform with a truly random function. By the standard ROM argument [BSBHR18], any distinguisher between Game 0 and Game 1 can be used to construct a collision-finder for H_{cr} , breaking (A3). Thus $|\varepsilon_0 - \varepsilon_1| \leq \text{negl}(\lambda)$.

Game 2. Apply the sumcheck extractor Σ (from A1/A2) to the prover's sumcheck messages $h_1, \dots, h_k$. The sumcheck is run on the single polynomial $P(j) = (AZ)_j \cdot (BZ)_j - u \cdot (CZ)_j - E[j]$, so Σ produces the multilinear extension $f_P : \{0, 1\}^k \rightarrow \mathbb{F}$ of the vector $P \in \mathbb{F}^{2^k}$ such that:

1. $\sum_{x \in \{0, 1\}^k} f_P(x) = 0$ (the claimed sum is 0).
2. $f_P(r) = \text{prod}_r$ for the random point $r = (r_1, \dots, r_k)$.

By Schwartz-Zippel, if $f_P(r) = 0$ for a random r , then f_P is identically zero with overwhelming probability. This means $P(j) = 0$ for all $j \in \{0, 1\}^k$, i.e., the relaxed R1CS equation $(AZ)_j \cdot (BZ)_j = u \cdot (CZ)_j + E[j]$ holds for every constraint. Therefore the vector (W, E) underlying the folded instance satisfies R_RELAXED.

The extractor E simply runs Σ on P^* 's sumcheck messages and outputs the witness (W, E) recovered from the extracted vector P (or, equivalently, from the MLE f_P evaluated at all Boolean hypercube points). Extraction succeeds with probability $\varepsilon_1 - \text{negl}(\lambda)$.

Combining the games: $\varepsilon_0 \leq \varepsilon_1 + \text{negl}(\lambda) \leq \Pr[E \text{ succeeds}] + \text{negl}(\lambda)$. Therefore E outputs a valid witness with probability $\varepsilon - \text{negl}(\lambda)$. $\square$

Remark 1 (SIS instantiation caveat). The theorem guarantees that a valid slim proof implies the existence of a witness satisfying `R_RELAXED`. For the *Pedersen* and *Hash* instantiations this is sufficient: the extractor outputs an arbitrary field-element witness and binding ensures it matches the commitments. For the *SIS* instantiation, post-quantum security additionally requires the extracted witness to be *short* (low-norm), i.e. $\|W\| \leq \beta$ for some bound β determined by the SIS parameters. Our current implementation computes $c = A \cdot W$ without norm checks, so Theorem 1 does *not* give a post-quantum guarantee for SIS under adversarial witnesses. This is a deliberate scoping decision: the SIS path is designed for batch proving where the prover is trusted during folding (Definition 1), and the slim-proof format itself is unchanged. Adding norm enforcement (e.g. via LatticeFold-style range proofs) is future work and requires no protocol changes.

Remark 2 (Quantum security of the protocol). Theorem 1 is proved in the *classical* random oracle model (ROM). For a quantum adversary two gaps remain. **(i) Fiat-Shamir in the QROM.** The sumcheck protocol is made non-interactive via the Fiat-Shamir transform. A quantum prover can query the random oracle in superposition, so the classical ROM argument does not directly apply. Fiat-Shamir is generally believed secure in the quantum ROM for constant-round protocols with negligible soundness error, and the sumcheck protocol satisfies both conditions, but a formal QROM proof for our exact instantiation is future work. **(ii) Quantum extractors.** Theorem 1 constructs a classical extractor E . A quantum adversary may produce a superposition of valid witnesses; whether a quantum extractor can extract a classical witness from the prover’s quantum state (the “collapsing” property) is standard for sumcheck but not verified here. Because of these two gaps we label NovaSlim’s post-quantum security as *conjectured* (Table 11) rather than *proven*. The commitment schemes (SIS and Hash) are themselves conjectured post-quantum, but the protocol-level guarantee requires closing the QROM and extraction gaps.

Remark 3 (What the slim proof loses). The full sumcheck proof includes HashPC opening proofs (truth tables) that allow *explicit* witness reconstruction: an auditor reads the truth tables and directly recovers (W, E) . The slim proof omits these openings. The theorem shows that *knowledge soundness* is preserved — a valid slim proof still implies the existence of a valid witness — but *extractability* changes from explicit (read the openings) to implicit (run the sumcheck extractor). For on-chain verification this distinction is irrelevant: the verifier only needs to know a valid witness exists. For off-chain audit, the full proof is published alongside the slim proof, and matching commitment hashes confirm they refer to the same witness.

Remark 4 (Commitment modularity). The proof is generic over the commitment scheme: the only property used is binding (A3). When switching from Pedersen to SIS or Hash, the extractor E is identical — only the instantiation of `Com` changes. This is a direct consequence of the `CommitmentScheme` trait abstraction in the implementation.

5.4 Equivocation and audit trails

A malicious prover could in principle claim the same commitment hash with different (W, E) values in different transactions. This is the *equivocation* problem. We mitigate it by including the commitment hashes in the deterministic Fiat-Shamir transcript, so the prover is bound to a specific (W, E) for each folding session. Cross-session equivocation requires finding a collision in the commitment hash or the transcript hash, both of which are computationally infeasible (by collision resistance of the hash functions).

5.5 What the slim proof keeps and what it strips

The slim verifier time (~ 0.2 ms for VRF, ~ 0.7 ms for Ed25519) is *pure sumcheck arithmetic*: challenge derivation, round-polynomial evaluation, and the final field-equation check. It

Table 2: Comparison of full vs. slim proof properties.

Property	Full proof	Slim proof
Soundness	Yes	Yes
Knowledge-soundness	Yes (explicit witness)	Yes (implicit witness)
On-chain size	~240 KiB	~0.4–1.5 KiB
Auditability	Full witness reconstruction	Commitment binding only
Trusted setup	None	None
Verifier time	~8 s (HashPC recompute)	~0.2 ms (sumcheck arithmetic only, excl. I/O)

excludes CBOR deserialization, transcript hashing, and commitment verification, which are done once per proof and add ~1–5 ms depending on proof size.

The slim proof is a deliberate compression that trades off-chain audit convenience for on-chain size, without compromising the core security property: a valid slim proof guarantees that a valid witness exists and is bound to the public commitments (Theorem 1).

5.6 Zero-knowledge properties

NovaSlim does *not* achieve zero-knowledge. We explain what the slim proof reveals, why this is acceptable, and how the full proof differs.

What the slim proof reveals. A slim proof $\pi = (h_1, \dots, h_k, r_1, \dots, r_k, prod_r)$ is public. From it, an adversary learns:

- **The sumcheck round polynomials** $h_1, \dots, h_k$. These reveal the prover’s claimed sums at each round, which encode partial information about the constraint products $(AZ)_j \cdot (BZ)_j - u \cdot (CZ)_j - E[j]$.
- **The Fiat-Shamir challenges** $r_1, \dots, r_k$. These are deterministic (derived from the transcript), so they reveal nothing beyond what the polynomials already expose.
- **The claimed product evaluation** $prod_r = P(r)$. This is the value of the per-constraint product polynomial at the random point r . By the Schwartz-Zippel lemma, it is consistent with any witness that satisfies the R1CS constraint at r , but it does not uniquely determine the witness.
- **The commitment hashes** Com_W, Com_E . These are deterministic commitments to the witness and error, but without the opening proofs (truth tables), they reveal nothing about the witness values.

What the slim proof does NOT reveal. The slim proof omits the HashPC opening proofs (truth tables), which would allow explicit witness reconstruction. Without these, an adversary cannot recover (W, E) from the slim proof alone — they would need to break the commitment binding (A3) or the collision resistance of the hash (A4).

Why this is acceptable. NovaSlim targets batch-proving scenarios where the prover is the data owner (e.g., a wallet proving transaction validity). The verifier needs assurance that a valid witness *exists*, not knowledge of the witness itself. The slim proof provides this guarantee (Theorem 1) without leaking the witness. In UTXO blockchains, the transaction already reveals the public input x (e.g., transaction hash, signature); the slim proof adds no new information about the private witness.

Full proof vs. slim proof. The full sumcheck proof includes the HashPC opening proofs (truth tables), which allow explicit witness reconstruction. An auditor with access to the full proof can recover (W, E) by reading the truth tables and checking consistency with the commitment hashes. This is the intended audit-trail use case: the full proof is

published alongside the slim proof, and matching commitment hashes confirm they refer to the same witness. The slim proof alone does not enable this reconstruction.

Comparison with SNARKs. Groth16 achieves zero-knowledge: the proof reveals nothing about the witness beyond the validity of the statement. NovaSlim sacrifices ZK for size: the slim proof is $\sim 0.4\text{--}1.5$ KiB vs. Groth16’s 192 B, but NovaSlim requires no trusted setup and works on a single curve. For the target use case (on-chain verification of batch-proved computations), the absence of ZK is acceptable because the prover is the data owner and the public input is already known to the verifier.

6 Implementation and evaluation

6.1 Implementation

We implement NovaSlim in Rust using the arkworks ecosystem. The codebase is curve-agnostic and organized into five modules:

- `circuit.rs`: Generic sparse R1CS parser (`.r1cs` / `.wtns`) for any arkworks prime field.
- `curve.rs`: Trait abstraction (`NovaCurve`) with concrete implementations for BLS12-381, BN254, Pallas, Vesta, Grumpkin, and Bandersnatch.
- `nifs.rs`: NIFS folding with deterministic Pedersen bases (hash-to-scalar), generic over `NovaCurve`.
- `sumcheck.rs`: Sumcheck prover/verifier + HashPC commitment scheme, generic over `NovaCurve`.
- `lib.rs`: Orchestration, CBOR codec, CLI bindings.

All cryptographic operations use constant-time implementations where available. The Pedersen commitment bases are derived deterministically from a public seed using BLAKE2b, eliminating the need for a trusted setup.

Parallelization. We use rayon for parallel cross-term computation (NIFS), parallel row-product evaluation (sumcheck), and parallel sumcheck compression prover (`eval_dense_mle`, round-internal sums, fold steps). Property tests verify that parallel and sequential paths produce byte-identical output.

6.2 Benchmark setup

We benchmark four families of real step circuits and one synthetic:

Table 3: Benchmark circuits.

Circuit	Family	Wires	Constraints	Steps
ed25519_verify_nova	Signature	7,658	7,724	255
poseidon_sponge_nova	Hash chain	636	633	255
poseidon_merkle_nova	Merkle path	639	639	32
sha256_step_nova (32 B)	Hash chain	31,273	31,584	32
sha256_step_big_nova (512 B)	Hash chain	62,225	58,973	32
vrf_verify_nova	VRF	9	9	254
synthetic_copy_state	Synthetic	$1 + 2w$	w	variable

The `ed25519_verify_nova` circuit is the Ed25519 key-ownership step used in prior work. The `poseidon_sponge_nova` circuit computes $state_out = Poseidon(state_in, domain)$ —

the same topology as Sonobe’s `hash_chain` benchmark, enabling a direct cross-system comparison. The SHA-256 circuits compute $state_out = SHA256(state_in)$ with varying input sizes (32 bytes and 512 bytes). The `vrf_verify_nova` circuit isolates protocol overhead with only 9 constraints. The `synthetic_copy_state` benchmark generates a trivial step circuit that copies its state and is parameterized by state width and step count; it matches the synthetic workload used in the Sonobe evaluation framework.

Measurements are from `cargo run -release -bin benchmark_nova` (real circuits) and `cargo run -release -bin benchmark_synthetic` (synthetic circuit) on a single machine (single core for baseline, 16-core for parallel).

6.3 Results

We present results in four parts: (1) proof sizes across all circuits; (2) end-to-end timing for ed25519, VRF, PoseidonSponge and PoseidonMerkle on all six curves; (3) commitment-scheme modularity on VRF; (4) SHA-256 scaling (small/medium/big) with parallel on/off; and (5) cross-system comparison with Sonobe.

6.3.1 Proof sizes

Table 4: Proof sizes across circuits (Pedersen commitment, single core).

Circuit	Constr.	NIFS bundle	Full sumcheck	Slim proof	Slim ver.
<code>vrf_verify_nova</code>	9	0.7 KiB	~1.6 KiB	~0.4 KiB	0.2 ms
<code>poseidon_sponge_nova</code>	633	0.4 KiB	~31 KiB	~0.6 KiB	0.4 ms
<code>poseidon_merkle_nova</code>	639	0.4 KiB	~31 KiB	~0.6 KiB	0.5 ms
<code>ed25519_verify_nova</code>	7,724	2.2 KiB	~240 KiB	~1.0 KiB	0.7 ms
<code>sha256_step_small_nova</code>	31,584	2.5 KiB	~1.2 MiB	~0.8 KiB	0.6 ms
<code>sha256_step_big_nova</code>	58,973	2.5 KiB	~2.3 MiB	~1.0 KiB	~1.0 ms

The slim proof depends only on $k = \log_2 n_{constraints}$, not on N or step width. VRF ($k = 4$) produces ~0.4 KiB proofs; SHA-256 big ($k = 16$) produces ~1.0 KiB — a 160–600× reduction from the full sumcheck proof, with **no opening proofs on-chain**. The NIFS bundle is $O(1)$ in both step count and circuit size (≤ 2.5 KiB for all circuits).

6.3.2 End-to-end performance across six curves

Table 5 shows fold, compress, and verify times for the four main circuits on all supported curves. BLS12-381 and BN254 use real circom circuits; Grumpkin, Pallas, and Vesta also use real circom circuits (circom 2.x supports their primes natively); Bandersnatch uses synthetic circuits (circom does not support the Bandersnatch scalar field).

Key observations. (1) Verification time scales logarithmically with $n_{constraints}$ and is independent of N : VRF verifies in 0.1–0.2 ms, Poseidon in 0.4–0.5 ms, ed25519 in 0.7 ms. (2) BN254 is consistently ~1.2–1.5× faster than BLS12-381 for folding, due to smaller scalar-field arithmetic. (3) Pasta curves (Pallas/Vesta) are fastest at 2.3–2.8 ms/step, followed by Grumpkin at 4.4 ms/step. The numbers shown are synthetic (state_width=4, 255 steps), which isolates pure field-arithmetic cost. (4) The slim proof size is **identical across all curves** for the same circuit because the sumcheck protocol is field-agnostic.

6.3.3 Commitment-scheme modularity

We benchmark the three commitment schemes on VRF (9 constraints, 254 steps, BLS12-381) to isolate commitment overhead:

Table 5: Timing results across circuits and curves (Pedersen, single core, baseline). † denotes synthetic benchmark (state_width matching circuit I/O count).

Circuit	Curve	Constr.	Steps	Fold	Fold/step	Compress	Ver. full	Ver. slim
vrf_verify_nova	BLS12-381	9	254	3.5 s	14 ms	0.04 s	0.05 s	0.2 ms
	BN254	9	254	2.2 s	9 ms	0.03 s	0.02 s	0.1 ms
	Grumpkin†	360	255	1.1 s	4.4 ms	0.03 s	0.02 s	0.1 ms
	Pallas†	360	255	0.59 s	2.3 ms	0.07 s	0.09 s	0.1 ms
	Vesta†	360	255	0.71 s	2.8 ms	0.01 s	0.01 s	0.1 ms
	Bandersnatch†	360	255	0.70 s	2.74 ms	0.09 s	0.09 s	0.3 ms
poseidon_sponge_nova	BLS12-381	633	255	46.2 s	181 ms	1.86 s	1.85 s	0.4 ms
	BN254	633	32	4.0 s	126 ms	1.09 s	1.06 s	0.5 ms
poseidon_merkle_nova	BLS12-381	639	32	10.0 s	314 ms	3.13 s	2.69 s	0.5 ms
ed25519_verify_nova	BLS12-381	7,724	255	138.5 s	543 ms	25.44 s	28.59 s	0.7 ms

Table 6: Commitment comparison (VRF circuit, BLS12-381, 254 steps).

Commitment	Fold total	Fold/step	Compress	Ver. full	Ver. slim	Slim proof
Pedersen	3.5 s	14 ms	0.04 s	0.05 s	0.2 ms	~0.4 KiB
SIS ($m = 4$)	0.2 s	1 ms	0.08 s	0.00 s	0.2 ms	~0.4 KiB
Hash	0.4 s	2 ms	0.05 s	0.00 s	0.2 ms	~0.4 KiB

SIS is $14\times$ **faster** than Pedersen for folding because the MSM cost is avoided entirely (inner products over $\mathbb{F}_r$). Hash is $7\times$ faster than Pedersen but derives coefficients on-the-fly via Blake2b. The slim proof is **independent of the commitment scheme** — the same ~ 0.4 KiB payload works for all three.

SIS parameter scaling. At cryptographic security ($m = 128$), fold time increases to 5.4 ms/step (BN254, synthetic) — still practical. The slim proof remains ~ 0.4 KiB regardless of m . Full verification increases because it re-derives and checks the SIS commitments, but remains fast enough for off-chain audit trails.

6.3.4 SHA-256 scaling: small, medium, and big

The SHA-256 circuits demonstrate how NovaSlim scales with constraint count. All three variants compute $state_out = SHA256(state_in)$ but differ in input size and padding:

Table 7: SHA-256 circuit family: constraint scaling.

Variant	State size	Constraints	$k = \log_2$
small	8 bytes	~29K	15
medium	32 bytes	~31K	15
big	32 bytes + 256-bit padding	~59K	16

Timing (BLS12-381, Pedersen, 32 steps):

Per-step costs are extrapolated from shorter runs (2–4 steps) after setup overhead stabilises. A full 32-step fold for SHA-256 big takes ~ 30 minutes total (~ 3 minutes with 16-core parallel compression). The SHA-256 circuits are the largest benchmarked; at this size, parallel sumcheck compression shows the biggest gains ($\sim 3\text{--}5\times$) because the sumcheck processes $n_{constraints}$ rows in parallel.

Memory and proof-size scaling. Memory consumption is $O(1)$ **in the step count**: only the current witness is kept in memory. Real circuits show slightly higher baseline

Table 8: SHA-256 timing (BLS12-381, Pedersen, 32 steps).

Variant	Constr.	Fold/step	Compress/step	Ver. slim	Slim proof
small	29K	~3,100 ms	~2,800 ms	0.6 ms	~0.8 KiB
medium	31K	~3,200 ms	~2,900 ms	0.7 ms	~0.8 KiB
big	59K	~5,600 ms	~5,200 ms	1.0 ms	~1.0 KiB

(~3–5 MiB for synthetic, higher for real circuits due to R1CS matrix loading), but still well under 100 MiB for 32 steps of the 59K-constraint SHA-256 big circuit. This is the critical advantage of folding over monolithic SNARKs: the prover never materializes the full $N \times C$ constraint system.

The slim proof size scales only with $k = \log_2 n_{constraints}$: SHA-256 small (~29K, $k = 15$) produces ~0.8 KiB proofs; SHA-256 big (~59K, $k = 16$) produces ~1.0 KiB — only a 25% increase for a $2\times$ increase in constraints.

6.3.5 Parallel speedup summary

Parallel speedup is most visible in the compress step (sumcheck), which processes $n_{constraints}$ rows in parallel. At small constraint counts ($\leq 7K$), thread overhead dominates and parallel mode is slightly slower.

Table 9: Parallel speedup (baseline vs. `-opt-parallel`, 16 cores).

Circuit	Constr.	Fold base	Fold par.	Compress base	Compress par.
vrf_verify_nova	9	3.5 s	4.1 s	0.04 s	0.08 s
poseidon_sponge_nova	633	2.1 s	2.1 s	2.09 s	1.99 s
poseidon_merkle_nova	639	10.0 s	10.9 s	3.13 s	2.72 s
ed25519_verify_nova	7,724	138.5 s	145.7 s	25.44 s	24.30 s

PoseidonMerkle (639 constraints) shows ~13% compress speedup. For SHA-256 big (~59K constraints), the parallel compress is expected to show ~3–5 $\times$ gains.

6.3.6 Cross-system comparison with Sonobe

We compare NovaSlim against Sonobe [0xP24], the leading folding library. Sonobe implements Nova with CycleFold (recursive IVC), requires a curve cycle (BN254 + Grumpkin), and compresses via Spartan + Groth16 decider. NovaSlim folds off-circuit (no in-circuit verification), uses a single curve, and compresses via sumcheck + HashPC (stripped for slim).

Table 10: Cross-system comparison with Sonobe.

Metric	Sonobe (Nova+CycleFold)	NovaSlim	Notes
Curve cycle	BN254 + Grumpkin	BLS12-381 only	Sonobe needs paired curves
Preprocess	7.7 s	0 s	NovaSlim has no IVC params
Keygen	7.6 s	0 s	NovaSlim has no proving key
Fold (32 steps)	111.5 s	7.6 s	14.7 $\times$ faster
Compress	N/A (part of decider)	1.96 s	Sumcheck compression
Verify	5.4 s (Groth16)	0.7 ms (slim)	7,700 $\times$ faster
Proof size	192 B (Groth16)	1.6 KiB (slim)	Different trust models
Trusted setup	Groth16 ceremony	None	NovaSlim is transparent
Security model	IVC (each step proven)	Batch (prover trusted)	Sonobe is strictly stronger

The comparison highlights different design trade-offs. The $14.7\times$ fold speedup reflects *both* the smaller circuit (633 real constraints vs. $\sim 65\text{K}$ augmented in Sonobe) *and* the architectural difference (off-circuit NIFS vs. in-circuit Spartan verification). Even normalised by constraint count, NovaSlim’s off-circuit fold is $\sim 10\times$ faster per constraint because it performs only field arithmetic (MSM for Pedersen, or matrix-vector products for SIS), while Sonobe’s in-circuit IVC must satisfy the entire Spartan verifier R1CS at each step. Sonobe’s recursive IVC provides a stronger security guarantee: each step proves the correctness of all previous steps. NovaSlim’s off-circuit folding trusts the prover during folding and relies on the final compressed proof for soundness. This is appropriate for batch proving where the prover is the wallet owner.

Sonobe achieves the smallest on-chain proof (192 B via Groth16) but requires two trusted-setup ceremonies, a curve cycle, and EVM precompiles that are unavailable on UTXO blockchains. NovaSlim produces a transparent 1.6 KiB proof that verifies natively via field arithmetic in sub-millisecond, making it compatible with, for example, Plutus-like eUTXO validators.

6.4 Discussion

Cost centers. The NIFS fold is dominated by two variable-base MSMs per step (Pedersen). The bases are deterministic, so these are really fixed-base MSMs. Precomputing window tables would reduce the 255-step fold from $\sim 46\text{ s}$ (PoseidonSponge, 633 constraints) to well under 10 s. This is a pure constant-factor optimization, not a protocol change.

Verify performance. Full verification ($< 2\text{ s}$ for PoseidonSponge) includes HashPC truth-table recomputation. Slim verification ($\sim 0.4\text{ ms}$) skips the openings and checks only the sumcheck protocol, making it practical for on-chain verification.

Deployable parameters. The SIS scheme scales to cryptographic security: at $m = 128$, fold time increases to 5.4 ms/step (BN254, synthetic) — still practical. However, full post-quantum security requires the witness vector v to be *short* (low-norm), which our current implementation does not enforce. This is a deliberate scoping decision: the SIS commitment is used for binding, not for the core security argument (Theorem 1), and switching to a norm-enforcing variant requires no protocol changes.

Comparison with STARKs. STARKs [BSBHR18] achieve post-quantum security with hash-only commitments but cannot produce sub-kilobyte proofs ($\sim 50\text{--}200\text{ KiB}$) because they cannot fold. NovaSlim’s folding scheme provides *$O(\text{step})$ memory scaling* and *sub-kilobyte proofs* simultaneously — essential for computations that exceed commodity RAM when expressed monolithically.

Table 11: Comparison of NovaSlim with STARKs. “Conjectured” PQ for NovaSlim means the commitment schemes (SIS/Hash) are conjectured post-quantum, but the protocol-level security proof (Theorem 1) is classical; see Remark 2.

Property	NovaSlim + SIS ($m = 128$)	NovaSlim + Hash	STARK (FRI-based)
Proof size	$\sim 0.4\text{--}1.5\text{ KiB}$ (slim)	$\sim 0.4\text{--}1.5\text{ KiB}$ (slim)	$\sim 50\text{--}200\text{ KiB}$
Verifier time	$\sim 0.2\text{ ms}$ (slim)	$\sim 0.4\text{ ms}$ (slim)	$\sim 10\text{--}100\text{ ms}$
Prover memory	$O(\text{step})$ (folding)	$O(\text{step})$ (folding)	$O(\text{total})$
PQ security	Conjectured (SIS)	Conjectured (Blake2b)	Proven (hash-only)
Trusted setup	None	None	None

6.5 The tiny-proof / trustless / PQ triad

NovaSlim demonstrates that three properties often treated as mutually exclusive — **sub-kilobyte proofs**, **no trusted setup**, and **post-quantum security** — can coexist in a

single system. This is achieved by decoupling the folding commitment from the sumcheck layer via a generic trait, so the commitment scheme can be swapped at runtime without changing the proof format or verifier.

Table 12: The tiny-proof / trustless / PQ triad.

System	Proof size	Trusted setup	Post-quantum path
Groth16	192 B	Required	No
STARK	50–200 KiB	None	Yes (hash-only)
Nova (original)	~10 KiB	None	No (curve cycle)
NovaSlim + Pedersen	~0.4–1.5 KiB	None	No (classical)
NovaSlim + SIS	~0.4–1.5 KiB	None	Yes (lattice)
NovaSlim + Hash	~0.4–1.5 KiB	None	Yes (hash-based)

NovaSlim is the first system we are aware of that achieves all three properties simultaneously, with two independent post-quantum paths.

6.6 Reproducibility

All benchmarks in this paper can be reproduced using the open-source implementation at <https://github.com/paweljakubas/nova-slim>. The repository includes:

- The full Rust implementation with `cargo test` (prover + CLI tests).
- Benchmark binaries (`benchmark_nova`, `benchmark_synthetic`) that reproduce all timing and memory measurements.
- Circom circuit source for all benchmarked circuits.

To reproduce the results in §6:

```
# Build in release mode
cargo build --release

# Run all tests
cargo test --release

# Six curves supported: bls12-381, bn254, grumpkin, bandersnatch,
#                      pallas, vesta
# Three commitments: pedersen, sis, hash
# Example: VRF on BLS12-381 with SIS commitment
cargo run --release --bin benchmark_nova -- \
  --curve bls12-381 --commitment sis --sis-param 4 \
  --circuit circom/VRF/vrf_verify_nova.r1cs \
  --steps /tmp/vrf_steps/

# Example: synthetic benchmark on Bandersnatch
cargo run --release --bin benchmark_synthetic -- \
  --curve bandersnatch --steps 255 --state-width 24

# Example: SHA-256 big with parallel compression
cargo run --release --bin benchmark_nova -- \
  --curve bls12-381 --opt-parallel \
  --circuit circom/Sha256Step/build_bls12381/sha256_step_big_nova.
  r1cs \
  --steps /tmp/sha256_big_steps/
```

An Aiken eUTXO verifier is available separately at github.com/paweljakubas/nova-slim/cardano/nova-slim-verifier.

All measurements were taken on a single machine with a 16-core CPU and 64 GiB RAM. The `-opt-parallel` flag enables rayon parallelism; without it, all benchmarks run on a single core.

7 Conclusion and future work

We presented NovaSlim, which makes three contributions to Nova-family IVC:

1. **Slim proofs.** We prove that polynomial commitment openings can be stripped from the on-chain payload while preserving knowledge soundness, reducing proof size from ~ 240 KiB to ~ 0.4 – 1.5 KiB (Theorem 1).
2. **Commitment-scheme modularity.** A generic `CommitmentScheme` trait with three instantiations (Pedersen, SIS, Hash), two of which are post-quantum. The security proof and verifier are generic over the commitment scheme.
3. **Empirical demonstration.** The combination of off-circuit folding, sumcheck+HashPC compression, and slim proofs produces sub-kilobyte transparent proofs that verify in under 1 ms, with two independent post-quantum commitment paths, all on a single standard curve.

The slim proof is ~ 0.4 – 1.5 KiB (depending only on the number of sumcheck rounds $k = \log_2 n_{\text{constraints}}$) — independent of the commitment scheme, step count, and step width — and fits comfortably inside, for example, Cardano’s 16 KiB transaction limit.

The next research directions:

1. **Fixed-base MSM optimization (Pedersen only).** The Pedersen bases are deterministic, so Straus’ algorithm or batched window precomputation could reduce fold times. This does not benefit SIS (matrix-vector multiplication) or Hash (on-the-fly derivation). Naive `FixedBase` tables require ~ 6 GB for 7,658 bases; a production implementation would need memory-efficient multi-scalar precomputation.
2. **In-circuit recursion.** Off-circuit folding trusts the prover during the fold phase: a malicious prover could corrupt the accumulator mid-way, and the error is only caught at the final compression. In-circuit recursion restores the “proof-carrying” property of classical IVC: each step includes a proof that the previous fold was performed correctly, so every intermediate accumulator is independently verifiable.

This is commitment-agnostic: Pedersen, SIS, and Hash can all be verified in-circuit, but the cost differs. Pedersen requires MSM verification (expensive in-circuit), SIS requires matrix-vector products (moderate), and Hash requires hash evaluations (cheapest). Thus recursion is most compelling for Hash and SIS, where the verifier circuit is small enough to keep per-step overhead manageable.

To verify Pedersen or Hash commitments in-circuit, the verifier must perform group or hash operations native in the R1CS field. This requires a *curve companion* whose base field equals the primary curve’s scalar field. For BLS12-381 the natural companion is Bandersnatch (defined over BLS12-381’s scalar field); for BN254 it is Grumpkin (the standard BN254/Grumpkin cycle used by Sonobe); for Pallas it is Vesta (the pasta cycle). Adding a companion curve to NovaSlim would enable true recursive IVC on any of the six supported curves without changing the proof format or verifier.

3. **Formal security in the QROM.** Theorem 1 assumes Fiat-Shamir soundness in the classical ROM (A2). A complete post-quantum guarantee requires a proof in the quantum ROM, where the adversary queries the random oracle in superposition, together with a quantum extractor for the sumcheck protocol (Remark 2). Both are believed to hold for constant-round protocols with negligible soundness error, but formal verification is future work.

Acknowledgments. We used AI-assisted coding tools for implementation and text-editing assistance. The authors take full responsibility for all content.

References

- [0xP24] 0xPARC / PSE. Sonobe: A modular folding library. <https://github.com/0xPARC/sonobe>, 2024.
- [BC24] Dan Boneh and Binyi Chen. Latticefold: A lattice-based folding scheme and its applications to succinct proof systems. IACR ePrint 2024/257, 2024.
- [BC25] Dan Boneh and Binyi Chen. Latticefold+: Faster, simpler, shorter lattice-based folding for succinct proof systems. IACR ePrint 2025/247, 2025.
- [BGH19] Sean Bowe, Jack Grigg, and Daira Hopwood. Recursive proof composition without a trusted setup. IACR ePrint 2019/1021, 2019.
- [BSBHR18] Eli Ben-Sasson, Iddo Bentov, Ynon Horesh, and Michael Riabzev. Scalable, transparent, and post-quantum secure computational integrity. IACR ePrint 2018/046, 2018.
- [Gro16] Jens Groth. On the size of pairing-based non-interactive arguments. In *EUROCRYPT 2016*, 2016. IACR ePrint 2016/260.
- [KS22] Abhiram Kothapalli and Srinath Setty. Supernova: Proving universal machine executions without universal circuits. IACR ePrint 2022/1758, 2022.
- [KS23] Abhiram Kothapalli and Srinath Setty. Cyclefold: Folding-scheme-based recursive arguments over a cycle of elliptic curves. IACR ePrint 2023/1192, 2023.
- [KS24] Abhiram Kothapalli and Srinath Setty. Hypernova: Recursive arguments for customizable constraint systems. In *CRYPTO 2024*, 2024. IACR ePrint 2023/573.
- [KST22] Abhiram Kothapalli, Srinath Setty, and Ioanna Tzialla. Nova: Recursive zero-knowledge arguments from folding schemes. In *CRYPTO 2022*, 2022. IACR ePrint 2021/370.
- [LFKN90] Carsten Lund, Lance Fortnow, Howard Karloff, and Noam Nisan. Algebraic methods for interactive proof systems. In *STOC 1990*, 1990.
- [Set20] Srinath Setty. Spartan: Efficient and general-purpose zksnarks without trusted setup. In *CRYPTO 2020*, 2020. IACR ePrint 2019/550.